

# AWS MLOps Project to Deploy a Classification Model [Banking]

## Business Overview

In a digital banking environment, the process of identifying the right customers for loan offers and financial products is important to increase business growth.

The focus of the project is to build a fully automated and scalable deployment pipeline that takes an ML model built with Flask and deploys it on Amazon EKS using Kubernetes. We recommend having a basic understanding of the previous project, [Build Classification Algorithms for Digital Transformation \[Banking\]](#), as this forms the foundation for the deployment process.

## Aim:

The project aims to deploy a classification model that identifies potential borrowers for banking services using AWS MLOps workflows. The project demonstrates how to automate every step from building and containerizing the ML model to deploying it on Amazon EKS through a CI/CD pipeline.

AWS CI/CD services such as CodeBuild, CodePipeline, ECR, and EKS are used to ensure a smooth and secure deployment process, which allows new model versions to be updated automatically. Additionally, Terraform is also used alongside UI to automate the creation of AWS Services.

## Prerequisite Project:

- [Build Classification Algorithms for Digital Transformation \[Banking\]](#)

## Prerequisites:

It is recommended to have a basic knowledge of the following:

1. Python
2. Flask
3. Docker
4. Basics of AWS
5. Basics of deployment
6. Terraform

## Data Description

The dataset consists of **two CSV files**, and each data file represents different aspects of customer information used for model training and inference.

- **Data 1:** It includes 5,000 records with 8 attributes, such as Age, CustomerSince, HighestSpend, ZipCode, HiddenScore, MonthlyAverageSpend, and Level, which provide insights into customer demographics and spending behavior.

- **Data 2:** It includes 5,000 records with 7 attributes, such as Mortgage, Security, FixedDepositAccount, InternetBanking, CreditCard, and LoanOnCard, which provide information on customer engagement with various financial products.

### **Tech Stack:**

→ **Language:** Python

→ **Libraries:** Flask, Gunicorn, Pandas, Scikit-learn

→ **Services:** AWS EKS, Amazon ECR, Load Balancer, AWS CodeBuild, AWS CodePipeline, AWS S3

→ **Tools:** Docker, Terraform

**Note:** We recommend monitoring AWS Services and deleting those that are not in use after project execution, whether they were created using the UI or Terraform.

### **Approach:**

In this project, the MLOps pipeline automates the deployment of a Flask-based machine learning application on AWS using Amazon EKS.

#### **1. Cluster Creation and Configuration**

- The Amazon EKS cluster was created to manage containerized workloads efficiently.
- Node groups were added to the cluster, and an OIDC connection was established to securely link AWS IAM with Kubernetes for authentication.

#### **2. Monitoring and Load Balancing**

- The Kubernetes dashboard was installed for real-time cluster monitoring.
- A Load Balancer Controller was configured to manage ingress traffic and balance requests across running containers.

#### **3. Deployment Configuration using Kubernetes YAMLs**

- Kubernetes manifests were created to define deployment behavior.
- There are separate YAML files for Deployment, Service, and Ingress that were configured to handle pod management and traffic routing in EKS.

#### **4. CI/CD Pipeline Configuration**

- Two GitHub repositories were created, one for the Flask-based ML application and another for the Kubernetes YAML configurations.
- Each repository contains a separate workflow file to automate build and deployment operations.
- The first workflow file handles the ML application's Docker image build, while the second manages its deployment on the EKS cluster through AWS CodeBuild.

#### **5. AWS CodePipeline**

- Two AWS CodePipelines were created, each linked with a dedicated CodeBuild project.

- The first pipeline integrates with the Flask application's GitHub repository, which automatically builds and pushes the Docker image to Amazon ECR whenever a new commit is made.
- The second pipeline monitors ECR for new images and, upon a new Docker image, triggers the deployment pipeline that updates the EKS cluster with the latest application version.

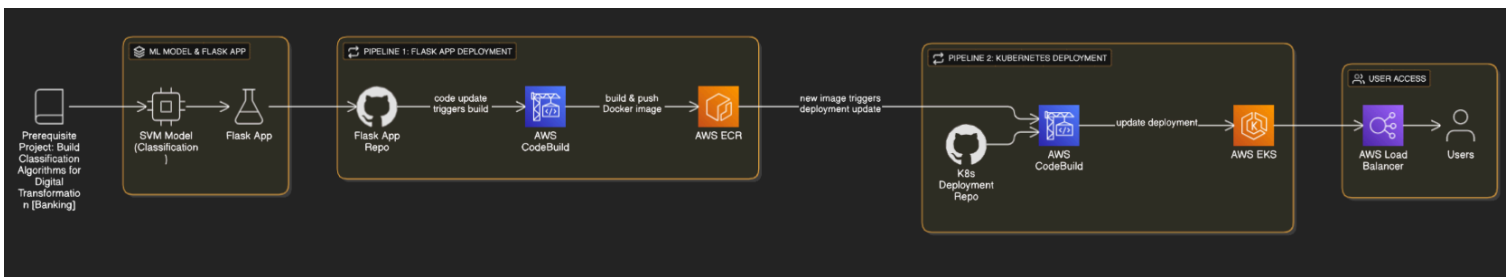
## 6. Application Load Balancing and Testing

- The Application Load Balancer (ALB) manages incoming API requests across active pods within the EKS cluster.

## 7. Continuous Integration and Infrastructure as Code (IaC)

- Terraform was used alongside UI to automate provisioning of services, including ECR, EKS, CodeBuild, and CodePipeline.
- Each new commit initiated a full deployment cycle to maintain continuous integration and delivery of the latest version of the application.

### Architecture Diagram:



### Key Takeaways:

- Learn how to design and deploy Flask-based ML applications on AWS EKS
- Learn how to create an EKS cluster, including node group setup, OIDC authentication, and IAM integration
- How to configure AWS Load Balancer Controller with EKS for load distribution?
- Learn how to set up the Kubernetes Dashboard to track pod health
- Learn how Amazon S3 integrates with the ML backend to store input datasets
- Learn how to automate the CI/CD workflow by connecting CodeBuild processes, ECR image updates, and EKS deployments
- Understanding of AWS CodeBuild projects for both pipelines to handle build, testing, and deployment operations seamlessly
- Creation of the first CodePipeline for building and pushing an image to ECR
- Understanding the setup of the second CodePipeline to automatically deploy Docker images to the EKS cluster
- Implementation of Infrastructure as Code (IaC) using Terraform to provision AWS resources